

6.1 Service Basics

Part of Services and Dependency Injection Module - Duration: 10-12 hours

1. What is a Service?

1.1 Definition

A service is a TypeScript class that encapsulates business logic, data operations, and functionality that can be shared across multiple components in an Angular application. Services provide a way to organize code that doesn't belong in components, making your application more maintainable and testable.

Think of a service as a utility provider - just like electricity or water services in your home that are available to all rooms without each room needing its own generator or well.

```
// Simple analogy: Library System
Application
├─ BookListComponent (displays books)
├─ BookDetailComponent (shows book details)
├─ FavoriteComponent (shows favorite books)
└─ BookService (manages all book data)
    ├─ getAllBooks()
    ├─ getBookById()
    ├─ addToFavorites()
    └─ searchBooks()
```

Output:

Service Concept: Instead of each component managing book data separately, a `BookService` handles all book-related operations centrally. Components request data from the service, ensuring consistency across the application.

1.1.1 Architecture Diagram

Visual comparison: application structure before and after introducing a centralized `BookService`.

Before (no service)

```

Application
├─ BookListComponent (manages books locally)
├─ BookDetailComponent (manages details locally)
└─ FavoriteComponent (manages favorites locally)

Each component keeps its own copy of book data, causing duplication and inconsistency.
  
```

After (with `BookService`)

```

Application
├─ BookListComponent → requests data from BookService
├─ BookDetailComponent → requests data from BookService
├─ FavoriteComponent → requests data from BookService
└─ BookService (centralized data & operations)
    ├── getAllBooks()
    ├── getBookById()
    ├── addToFavorites()
    └── searchBooks()

Components ask BookService for data and actions; a single source of truth prevents duplication.
  
```

1.2 Why Use Services?

Services address several architectural challenges in Angular applications:

Problem	Service Solution
Code Duplication	Write business logic once, use it in multiple components
Component Bloat	Keep components focused on view logic, move business logic to services
Data Consistency	Centralized data management ensures all components work with the same data
Testability	Test business logic independently from UI components
Maintainability	

Problem	Service Solution
	Update logic in one place, changes reflect everywhere
Reusability	Share functionality across different modules and components

1.3 Component vs Service: What Goes Where?

Understanding what belongs in a component versus what belongs in a service is fundamental to building well-structured Angular applications.

Component Responsibilities	Service Responsibilities
Display data and UI elements	Fetch and manage data
Handle user interactions (clicks, inputs)	Perform calculations and transformations
Bind data to the template	Communicate with backend APIs
Manage component-specific state	Share data between components
React to view events	Implement business rules and validation
Control component lifecycle	Cache data and manage application state

⚡ Important:

The general rule: **Components** handle the "what to display", **services** handle the "how to get it".

2. Creating Services

2.1 Manual Service Creation

You can create a service manually by creating a TypeScript class file. Let's create a simple service that manages a list of products.

Step 1: Create the Service File

```
// product.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ProductService {

  private products = [
    { id: 1, name: 'Laptop', price: 999 },
    { id: 2, name: 'Mouse', price: 29 },
    { id: 3, name: 'Keyboard', price: 79 }
  ];

  getAllProducts() {
    return this.products;
  }

  getProductById(id: number) {
    return this.products.find(product => product.id === id);
  }

  addProduct(product: any) {
    product.id = this.products.length + 1;
    this.products.push(product);
  }
}
```

Output:

Service Structure:

- `@Injectable()` decorator marks this class as a service
- `providedIn: 'root'` makes it available application-wide
- `products` array stores the data
- Methods expose operations on the data

2.2 Using Angular CLI to Create Services

Angular CLI provides a convenient command to generate services with proper structure and boilerplate code.

Command Syntax

```
ng generate service service-name
// or shorthand
ng g s service-name
```

Example: Creating a Logger Service

```
ng generate service services/logger
```

Output:

CLI Output:

```
CREATE src/app/services/logger.service.spec.ts (357 bytes)
CREATE src/app/services/logger.service.ts (135 bytes)
```

This command creates two files:

- `logger.service.ts` - The service class file
- `logger.service.spec.ts` - The unit test file for the service

Generated Service File

```
// logger.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class LoggerService {
```

```
constructor() { }  
  
}
```

 Note:

The CLI automatically adds the `@Injectable()` decorator and sets `providedIn: 'root'`, making the service ready to use across the application.

Adding Functionality to the Logger Service

```
// logger.service.ts  
import { Injectable } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class LoggerService {  
  
  private logs: string[] = [];  
  
  constructor() { }  
  
  log(message: string): void {  
    const timestamp = new Date().toLocaleTimeString();  
    const logEntry = `[${timestamp}] ${message}`;  
    this.logs.push(logEntry);  
    console.log(logEntry);  
  }  
  
  error(message: string): void {  
    const timestamp = new Date().toLocaleTimeString();  
    const logEntry = `[${timestamp}] ERROR: ${message}`;  
    this.logs.push(logEntry);  
    console.error(logEntry);  
  }  
  
  getLogs(): string[] {  
    return this.logs;  
  }  
  
  clearLogs(): void {
```

```

    this.logs = [];
    console.clear();
  }
}

```

Output:

Logger Service Features:

- `log()` - Records informational messages
- `error()` - Records error messages with ERROR prefix
- `getLogs()` - Returns all stored log entries
- `clearLogs()` - Clears all logs from memory

2.3 Manual vs CLI: When to Use Which?

Approach	Advantages	When to Use
Manual Creation	• Full control over file structure • No unnecessary boilerplate • Learn the internal structure	• Learning Angular fundamentals • Creating simple utility services • Custom file organization needed
Angular CLI	• Consistent file naming • Automatic test file generation • Follows Angular conventions • Faster development	• Production applications • Team projects (consistency) • When test files are needed • Standard service structure

3. Service Implementation Examples

3.1 Data Management Service

Let's create a practical service that manages a task list - a common requirement in many applications.

```
// task.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class TaskService {

  private tasks = [
    { id: 1, title: 'Learn Angular Services', completed: false },
    { id: 2, title: 'Build Todo Application', completed: false }
  ];

  private nextId = 3;

  constructor() { }

  // Get all tasks
  getTasks() {
    return this.tasks;
  }

  // Get a single task by ID
  getTaskById(id: number) {
    return this.tasks.find(task => task.id === id);
  }

  // Add a new task
  addTask(title: string) {
    const newTask = {
      id: this.nextId++,
      title: title,
      completed: false
    };
    this.tasks.push(newTask);
    return newTask;
  }

  // Toggle task completion status
  toggleTask(id: number) {
    const task = this.getTaskById(id);
    if (task) {
      task.completed = !task.completed;
    }
  }
}
```



```
// Delete a task
deleteTask(id: number) {
  const index = this.tasks.findIndex(task => task.id === id);
  if (index !== -1) {
    this.tasks.splice(index, 1);
  }
}

// Get completed tasks count
getCompletedCount(): number {
  return this.tasks.filter(task => task.completed).length;
}

// Get pending tasks count
getPendingCount(): number {
  return this.tasks.filter(task => !task.completed).length;
}
}
```

Output:

TaskService Methods:

- `getTasks()` - Returns all tasks
- `getTaskById(id)` - Returns a specific task
- `addTask(title)` - Creates and adds a new task
- `toggleTask(id)` - Marks task as complete/incomplete
- `deleteTask(id)` - Removes a task from the list
- `getCompletedCount()` - Returns number of completed tasks
- `getPendingCount()` - Returns number of pending tasks

3.2 Using the Service in a Component

Now let's see how to use the `TaskService` in a component. First, we need to inject the service into the component.

```
// task-list.component.ts
import { Component, OnInit } from '@angular/core';
import { TaskService } from '../task.service';
```

```
@Component({
  selector: 'app-task-list',
  templateUrl: './task-list.component.html',
  styleUrls: ['./task-list.component.css']
})
export class TaskListComponent implements OnInit {

  tasks: any[] = [];
  newTaskTitle: string = '';

  constructor(private taskService: TaskService) {
    // TaskService is injected through the constructor
  }

  ngOnInit(): void {
    // Load tasks when component initializes
    this.tasks = this.taskService.getTasks();
  }

  onAddTask(): void {
    if (this.newTaskTitle.trim()) {
      this.taskService.addTask(this.newTaskTitle);
      this.newTaskTitle = ''; // Clear input
      this.tasks = this.taskService.getTasks(); // Refresh list
    }
  }

  onToggleTask(taskId: number): void {
    this.taskService.toggleTask(taskId);
  }

  onDeleteTask(taskId: number): void {
    this.taskService.deleteTask(taskId);
    this.tasks = this.taskService.getTasks(); // Refresh list
  }

  getCompletedCount(): number {
    return this.taskService.getCompletedCount();
  }

  getPendingCount(): number {
    return this.taskService.getPendingCount();
  }
}
```

Output:

Service Injection:

- Service is injected via constructor: `constructor(private taskService: TaskService)`
- The `private` keyword creates a class property automatically
- Angular's dependency injection system provides the service instance
- Component methods call service methods to perform operations

3.3 Component Template

```
<!-- task-list.component.html -->
<div class="task-container">
  <h2>Task Manager</h2>

  <!-- Task Statistics -->
  <div class="task-stats">
    <p>Pending: {{ getPendingCount() }}</p>
    <p>Completed: {{ getCompletedCount() }}</p>
  </div>

  <!-- Add Task Form -->
  <div class="add-task">
    <input
      type="text"
      [(ngModel)]="newTaskTitle"
      placeholder="Enter new task"
      (keyup.enter)="onAddTask()">
    <button (click)="onAddTask()">Add Task</button>
  </div>

  <!-- Task List -->
  <div class="task-list">
    <div *ngFor="let task of tasks" class="task-item">
      <input
        type="checkbox"
        [checked]="task.completed"
        (change)="onToggleTask(task.id)">

      <span [class.completed]="task.completed">
        {{ task.title }}
      </span>
    </div>
  </div>
```

```
<button (click)="onDeleteTask(task.id)">Delete</button>
</div>
</div>
</div>
```

Output:

Browser Display:

Task Manager

Pending: 2

Completed: 0

Add Task

☐

Learn Angular Services

Delete

☐

Build Todo Application

Delete

3.4 Calculation Service Example

Services are perfect for encapsulating calculation logic that might be used across multiple components.

```
// calculator.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CalculatorService {

  constructor() { }

  add(a: number, b: number): number {
```

```
    return a + b;
  }

  subtract(a: number, b: number): number {
    return a - b;
  }

  multiply(a: number, b: number): number {
    return a * b;
  }

  divide(a: number, b: number): number {
    if (b === 0) {
      throw new Error('Cannot divide by zero');
    }
    return a / b;
  }

  percentage(value: number, total: number): number {
    if (total === 0) {
      return 0;
    }
    return (value / total) * 100;
  }

  average(numbers: number[]): number {
    if (numbers.length === 0) {
      return 0;
    }
    const sum = numbers.reduce((acc, num) => acc + num, 0);
    return sum / numbers.length;
  }
}
```

Using Calculator Service in a Component

```
// grade-calculator.component.ts
import { Component } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-grade-calculator',
  templateUrl: './grade-calculator.component.html'
})
export class GradeCalculatorComponent {
```

```
scores: number[] = [85, 92, 78, 95, 88];
totalMarks: number = 100;
averageScore: number = 0;
percentage: number = 0;

constructor(private calculator: CalculatorService) {
  this.calculateResults();
}

calculateResults(): void {
  // Calculate average score
  this.averageScore = this.calculator.average(this.scores);

  // Calculate percentage
  this.percentage = this.calculator.percentage(
    this.averageScore,
    this.totalMarks
  );
}

addScore(score: number): void {
  this.scores.push(score);
  this.calculateResults();
}
```

Output:

Calculated Results:

Scores: [85, 92, 78, 95, 88]

Average Score: 87.6

Percentage: 87.6%

4. The @Injectable() Decorator

4.1 Understanding @Injectable()

The `@Injectable()` decorator is essential for Angular services. It marks a class as available for dependency injection and provides configuration for how the service should be provided.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class MyService {
  // Service code
}
```

4.2 Decorator Metadata Properties

4.2.1 providedIn: 'root'

This is the most common configuration. It makes the service available throughout the entire application as a singleton.

```
@Injectable({
  providedIn: 'root'
})
export class GlobalService {
  // Available everywhere in the application
  // Only one instance exists
}
```

Output:

Benefits of providedIn: 'root':

- Service is available application-wide
- Only one instance is created (singleton)
- Tree-shakable (removed if not used)
- No need to add to providers array in modules

4.2.2 Why is @Injectable() Required?

The `@Injectable()` decorator serves several important purposes:

Purpose	Explanation
Metadata Generation	Generates metadata that Angular needs for dependency injection
Service Dependencies	Enables the service to have its own dependencies injected
Configuration	Specifies how and where the service should be provided
Tree-shaking	Allows unused services to be removed from production builds

⚡ Important:

Always include `@Injectable()` decorator on every service class, even if it doesn't inject other services. This is a best practice and prevents errors if you later need to inject dependencies.

4.3 Service with Dependencies

Services can depend on other services. The `@Injectable()` decorator enables this by allowing Angular to inject dependencies into the service constructor.

```
// user-data.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class UserDataService {

  private currentUser = {
    name: 'John Doe',
    role: 'Admin'
  };

  getCurrentUser() {
    return this.currentUser;
  }
}
```



```
isAdmin(): boolean {  
  return this.currentUser.role === 'Admin';  
}  
}
```

```
// notification.service.ts  
import { Injectable } from '@angular/core';  
import { UserDataService } from './user-data.service';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class NotificationService {  
  
  private notifications: string[] = [];  
  
  // UserDataService is injected here  
  constructor(private userDataService: UserDataService) { }  
  
  addNotification(message: string): void {  
    const user = this.userDataService.getCurrentUser();  
    const notification = `[${user.name}] ${message}`;  
    this.notifications.push(notification);  
    console.log(notification);  
  }  
  
  getNotifications(): string[] {  
    return this.notifications;  
  }  
}
```

Output:

Service Dependency Chain:

Component → NotificationService → UserDataService

- NotificationService depends on UserDataService
- Angular automatically provides UserDataService when NotificationService is created
- Both services are singletons (one instance each)

4.4 Example: Service Using Another Service

```
// Component using NotificationService
import { Component } from '@angular/core';
import { NotificationService } from './notification.service';

@Component({
  selector: 'app-dashboard',
  template: `
    <div>
      <h2>Dashboard</h2>
      <button (click)="sendNotification()">Send Notification</button>

      <div class="notifications">
        <h3>Notifications:</h3>
        <p *ngFor="let notif of getNotifications()">{{ notif }}</p>
      </div>
    </div>
  `,
})
export class DashboardComponent {

  constructor(private notificationService: NotificationService) { }

  sendNotification(): void {
    this.notificationService.addNotification('Task completed successfully');
  }

  getNotifications(): string[] {
    return this.notificationService.getNotifications();
  }
}
```

Output:

Output After Clicking "Send Notification":

Notifications:

[John Doe] Task completed successfully

5. Service Best Practices

5.1 Single Responsibility Principle

Each service should have one clear responsibility. Don't create "God services" that do everything.

 Tip:

Good Practice:

- `UserService` - Manages user data
- `AuthService` - Handles authentication
- `LoggerService` - Manages logging

Bad Practice:

- `AppService` - Does everything (user, auth, logging, data, etc.)

5.2 Naming Conventions

Follow Angular naming conventions for services:

- Service file: `service-name.service.ts`
- Service class: `ServiceNameService`
- Use descriptive names that indicate the service purpose

```
// Good examples
product.service.ts → ProductService
user-authentication.service.ts → UserAuthenticationService
data-storage.service.ts → DataStorageService

// Bad examples
product.ts → Product (not clear it's a service)
myservice.service.ts → MyService (too generic)
ProductServ.service.ts → ProductServ (abbreviated)
```

5.3 Keep Services Stateless When Possible

While services can maintain state, consider whether state management is necessary. Stateless services are easier to test and understand.

```
// Stateless service (preferred when possible)
@Injectable({ providedIn: 'root' })
export class MathUtilityService {
  calculateTax(amount: number, rate: number): number {
    return amount * rate;
  }

  formatCurrency(amount: number): string {
    return `$$${amount.toFixed(2)}`;
  }
}

// Stateful service (when state management is needed)
@Injectable({ providedIn: 'root' })
export class ShoppingCartService {
  private items = []; // State

  addItem(item: any): void {
    this.items.push(item);
  }

  getItems() {
    return this.items;
  }
}
```

5.4 Service Documentation

Document your services clearly, especially public methods that will be used by components.

```
@Injectable({ providedIn: 'root' })
export class ProductService {

  /**
   * Retrieves all products from the data source
   * @returns Array of product objects
   */
}
```

```
getAllProducts() {  
  return this.products;  
}  
  
/**  
 * Finds a product by its unique identifier  
 * @param id - The product ID to search for  
 * @returns Product object if found, undefined otherwise  
 */  
getProductById(id: number) {  
  return this.products.find(p => p.id === id);  
}  
}
```

6. Common Service Patterns

6.1 Configuration Service

```
@Injectable({ providedIn: 'root' })  
export class ConfigService {  
  
  private config = {  
    appName: 'Task Manager Pro',  
    version: '1.0.0',  
    maxTasksPerUser: 100,  
    enableNotifications: true  
  };  
  
  getConfig() {  
    return this.config;  
  }  
  
  getAppName(): string {  
    return this.config.appName;  
  }  
  
  getMaxTasks(): number {  
    return this.config.maxTasksPerUser;  
  }  
  
  areNotificationsEnabled(): boolean {
```

```
    return this.config.enableNotifications;
  }
}
```

6.2 Utility Service

```
@Injectable({ providedIn: 'root' })
export class UtilityService {

  constructor() { }

  // Generate random ID
  generateId(): string {
    return Math.random().toString(36).substr(2, 9);
  }

  // Format date to readable string
  formatDate(date: Date): string {
    const options: Intl.DateTimeFormatOptions = {
      year: 'numeric',
      month: 'long',
      day: 'numeric'
    };
    return date.toLocaleDateString('en-US', options);
  }

  // Capitalize first letter
  capitalizeFirst(text: string): string {
    if (!text) return '';
    return text.charAt(0).toUpperCase() + text.slice(1).toLowerCase();
  }

  // Check if string is empty or only whitespace
  isEmpty(text: string): boolean {
    return !text || text.trim().length === 0;
  }
}
```

6.3 Data Transformation Service

```
@Injectable({ providedIn: 'root' })
export class DataTransformService {

  constructor() { }

  // Transform array to key-value pairs
  arrayToKeyValue(arr: any[], keyField: string, valueField: string) {
    const result: any = {};
    arr.forEach(item => {
      result[item[keyField]] = item[valueField];
    });
    return result;
  }

  // Group array items by a property
  groupBy(arr: any[], property: string) {
    return arr.reduce((grouped, item) => {
      const key = item[property];
      if (!grouped[key]) {
        grouped[key] = [];
      }
      grouped[key].push(item);
      return grouped;
    }, {});
  }

  // Filter unique values from array
  getUniqueValues(arr: any[]): any[] {
    return [...new Set(arr)];
  }
}
```

7. Summary

⚡ Important:

Key Takeaways:

1. What Are Services?

- TypeScript classes that encapsulate business logic and data operations

- Provide shared functionality across components
- Separate concerns from component view logic

2. Creating Services:

- Manual: Create .ts file with @Injectable() decorator
- CLI: `ng generate service service-name`
- Always include @Injectable() decorator

3. Using Services:

- Inject via component constructor
- Access methods and properties through injected instance
- Services can depend on other services

4. Best Practices:

- Single responsibility per service
- Use `providedIn: 'root'` for application-wide services
- Follow naming conventions (name.service.ts)
- Keep services focused and well-documented

8. Practice Exercise

Note:

Exercise: Create a Student Management Service

Create a `StudentService` that manages student records with the following requirements:

1. Data Structure:

- Each student should have: id, name, age, grade, isActive
- Initialize with at least 3 sample students

2. Methods to Implement:

- `getAllStudents()` - Return all students
- `getActiveStudents()` - Return only active students

- `getStudentById(id)` - Return specific student
- `addStudent(student)` - Add new student
- `updateStudentGrade(id, newGrade)` - Update student's grade
- `toggleStudentStatus(id)` - Toggle active/inactive status
- `getAverageGrade()` - Calculate average grade of all students

3. Component Usage:

- Create a `StudentListComponent` that uses the service
- Display all students in a table
- Show student count and average grade
- Add buttons to toggle status and update grades

Next: 6.2 Dependency Injection - Understanding Angular's dependency injection system and hierarchical injectors